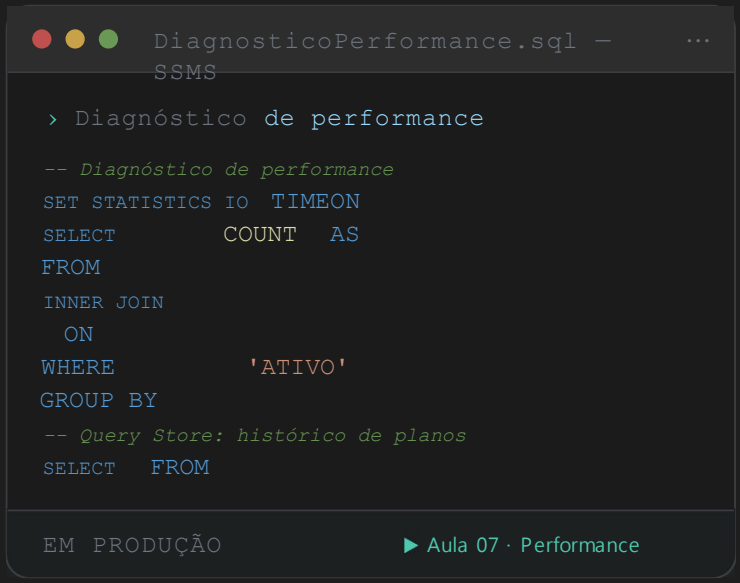
BANCO DE DADOS · AULA 07 MSSQL · T-SQL

Otimização de Consultas e Índices

Planos de Execução, Índices e Técnicas de Otimização · Ferramentas de Monitoramento e Análise Monitoramento e Análise de Desempenho.

PLANOS DE EXECUÇÃO ÍNDICES & ESTATÍSTICAS PERFORMANCE & OTIMIZAÇÃO



SUMÁRIO

Agenda da Aula

Dois blocos cobrindo conceito estrutural e operação prática de otimização.

DURAÇÃO TOTAL

105′

7.1

Planos de Execução, Índices e Técnicas de Otimização

- ›Como o otimizador funciona
- ›Plano de execução (estimado vs real)
- ›Operadores comuns (Scan, Seek, Join, Sort)
- ›Tipos de índice (Clustered, Nonclustered, Filtered, Columnstore)
- ›Estatísticas e histograma
- ›Técnicas set-based
- ›Hints (RECOMPILE, OPTIMIZE FOR, INDEX)
- ›SARGability
- ›Casos práticos

7.2

Ferramentas de Monitoramento e Análise de Desempenho

- ›DMVs essenciais
- ›Statistics IO/TIME
- ›Query Store
- ›Extended Events
- ›Wait statistics
- ›Activity Monitor
- ›Performance Monitor
- ›Profiler vs XEvents
- ›Checklist de diagnóstico · Casos reais

BLOCO 7.1 / 2

7.1

~ 55 MIN · 9 TÓPICOS

CHAPTER

Planos de Execução, Índices e Técnicas de Otimização de Consultas

O motor invisível — como o otimizador escolhe como executar suas queries e como ajudá-lo. como ajudá-lo.

01 → Como o otimizador funciona

02 → Plano de execução

03 → Operadores

04 → Tipos de índice

05 → Estatísticas

06 → Técnicas set-based

07 → Hints

08 → SARGability

09 → Casos práticos

FUNÇÃO INTERNA

Como o Otimizador Funciona

Quando você envia uma query T-SQL, ela passa por um pipeline de 4 fases antes de executar: Parsing (análise sintática), Binding (resolução de nomes/tipos), Optimization (escolha da estratégia de execução), e Execution (execução efetiva). O otimizador é a fase mais complexa — gera múltiplas alternativas e escolhe a de menor custo estimado.

O otimizador é cost-based: cada operação tem custo estimado (CPU, I/O, memória) baseado em estatísticas. Com custos, ele compara estratégias e escolhe a mais barata. Por isso estatísticas desatualizadas geram planos ruins — o otimizador decide com dados errados.

CARACTERÍSTICAS

- ›Baseado em custo (cost-based)
- ›Gera múltiplas alternativas (até milhares)
- ›Poda por custo acumulado (não explora tudo)
- ›Estatísticas são entrada crítica
- ›Índices são considerados como alternativas
- ›Cache de planos (reutilização)
- ›Pode paralelizar automaticamente

⚠ O QUE PODE DAR ERRADO

Estatísticas desatualizadas · Parameter sniffing (visto na Aula 02) · Funções não-SARGable · Fragmentação de índices · Tipos implícitos (conversion).

PIPELINE DE EXECUÇÃO

PARSE — Análise sintática
Verifica sintaxe T-SQL, gera árvore de parse.

BIND / ALGEBRIZE — Resolução
Valida nomes de objetos, colunas, tipos. Verifica permissões.

OPTIMIZE — Otimização
Gera alternativas, estima custos, escolhe plano mais barato.

EXECUTE — Execução
Executa o plano escolhido, retorna resultados.

✦ CUSTO ESTIMADO

I/O	ler páginas do disco	PRINCIPAL
CPU	processamento	SECUNDÁRIO
MEM	grants	TERCIÁRIO
Σ	SubTreeCost	TOTAL

ⓘ O plano escolhido é armazenado em cache (procedure cache) para reuso futuro.

LEITURA DE PLANOS

Plano de Execução — Estimado vs Real

O SQL Server oferece duas formas de visualizar o plano de execução: estimado (gerado sem executar a sem executar a query) e real (gerado após execução, com métricas reais). Ler ambos é skill essencial. essencial.

TIPOS DE PLANO

- ›Estimado — gerado sem executar. Mostra o que o otimizador acha que vai acontecer. acontecer. Disponível no SSMS via Ctrl+L.
- ›Real — gerado após execução. Mostra o que realmente aconteceu (linhas, tempo, I/O). Via Ctrl+M.
- ›Texto — SET SHOWPLAN_TEXT ON
- ›XML — SET SHOWPLAN_XML ON

QUANDO USAR CADA UM

- ›Estimado: queries muito longas, produção (não executa)
- ›Real: queries rápidas, dev (executa e captura métricas)
- ›XML: para ferramentas de análise

MÉTRICAS IMPORTANTES

- ›SubTreeCost — custo acumulado
- ›EstimateRows — linhas estimadas
- ›ActualRows — linhas reais (apenas real)
- ›EstimateIO / EstimateCPU — custos parciais
- ›Executions — quantas vezes o operador executou
- ›Discrepância Estimado vs Real = estatística ruim

SINAIS DE PROBLEMA

EstimateRows = 1, ActualRows = 1M (estatística desatualizada) · Table Scan em tabela grande (sem índice) · Key Lookup frequente (índice não cobre) · Sort com custo alto · Spool (recomputação).

PlanoEstimadoVsReal.sql · T-SQL

```
-- 1. Habilitar plano de execução estimado (não executa)
SET SHOWPLAN_XML ON
GO
SELECT       COUNT   AS
 FROM
INNER JOIN
    ON
 WHERE          'ATIVO'
 GROUP BY
GO
 SET SHOWPLAN_XML OFF
GO

-- 2. Habilitar estatísticas de I/O e tempo
SET STATISTICS IO ON
SET STATISTICS TIME ON
GO

-- 3. Habilitar plano real (via SSMS: Ctrl+M antes de executar)
-- Ou via XML:
SET STATISTICS XML ON
GO
SELECT       COUNT   AS
 FROM
INNER JOIN
    ON
 WHERE          'ATIVO'
 GROUP BY
GO
SET STATISTICS XML OFF
GO

-- Output do STATISTICS IO:
-- Table 'Vendas'. Scan count 5, logical reads 1234,
--   physical reads 12, read-ahead reads 1100, ...
-- Table 'Clientes'. Scan count 1, logical reads 42,
--   physical reads 0, read-ahead reads 40, ...

-- Output do STATISTICS TIME:
-- SQL Server Execution Times:
--   CPU time = 47 ms,  elapsed time = 52 ms.

-- 4. Consultar planos em cache
SELECT TOP 20
   OBJECT_NAME           AS
                  AS
                                    AS
                       1000                 AS
                       1000                  AS
              AS
 FROM
CROSS APPLY
CROSS APPLY
 WHERE        DB_ID
 ORDER BY                  DESC
 GO
```

OPERADORES · 10

Operadores Comuns do Plano de Execução

Conhecer os operadores é pré-requisito para ler qualquer plano de execução.

✓OK ⚠WARN ✗RISK

Operador	O que fazfunção	Quando é bomuse case	Quando é problemawatch out	Sinaisdiagnóstico
Table Scan/ Clustered Index Scan	Lê toda a tabela	Tabela pequena (<10k) ou SELECT * sem WHERE	Tabela grande com WHERE que deveria usar Seek	✗ Falta índice na coluna filtrada
Index Seek/ Clustered Index Seek	Lê apenas linhas específicas via índice	Sempre que há WHERE com índice	Raramente (mas pode ser seletividade ruim)	✓ Operador esperado em queries filtradas
Key Lookup	Busca colunas não cobertas pelo índice	Aceitável em tabelas pequenas	Em tabelas grandes, por linha — multiplica I/O	⚠ Adicionar INCLUDE no índice resolve
Hash MatchJoin / Aggregate	Agregação/Join em memória via hash table	Grandes conjuntos não ordenados	Pequenos conjuntos onde Nested Loop seria melhor	⚠ Memory grant alto, possível spill em tempdb
Merge Join	Join entre dois conjuntos ordenados	Ambos os lados já ordenados (índice clustered)	Quando exige Sort adicional	✓ Eficiente se dados já ordenados
Nested LoopsJoin	Para cada linha outer, busca inner	Outer pequeno + inner indexado	Outer grande → executa N vezes inner	✓ Melhor para conjuntos pequenos
Sort	Ordena o conjunto	Quando necessário (ORDER BY)	Em grandes conjuntos, gasta memória e tempdb	⚠ Adicionar índice na coluna de ordenação elimina Sort
Filter	Aplica predicado não-SARGable	Em casos simples	Quando força Scan em vez de Seek	✗ Refatorar WHERE para SARGable
Spool	Salva resultado intermediário em tempdb	Quando reutiliza dados repetidamente	Quando indicaria recompute desnecessário	⚠ Investigar se pode ser eliminado
ParallelismGather Streams	Combina resultados de múltiplas CPUs	Queries pesadas em servidor multi-core	Em queries simples, overhead desnecessário	⚠ MAXDOP controla nível de paralelismo

Regra prática: o operador mais caro `(mais grosso visualmente no plano gráfico)` é geralmente onde está o problema.

COMPARATIVO · 5 TIPOS

Tipos de Índice no SQL Server

Cada tipo serve a um propósito — escolher bem impacta performance diretamente.

✓OK ⚠WARN ✗RISK

Característica	Clusteredordem física	NonclusteredB-Tree separada	Filteredsubset	ColumnstoreOLAP / DW	In-MemoryHekaton
Estrutura	B-Tree, define ordem física	B-Tree separada da tabela	Nonclustered com WHERE	Columnar comprimido	Estrutura in-memory
Quantos por tabela	1 (um só)	Até 999	Até 999	1 clustered columnstore	Tabela inteira na RAM
Ideal para	PK, range queries	Busca por coluna não-PK	Subconjunto (Status= "ATIVO")	OLAP / DW / analytics	OLTP de altíssima carga
Performance leitura	✓ Excelente em range	✓ Excelente em lookup	✓✓ Excelente (subset menor)	✓✓ Excelente em agregação	✓✓✓ Excelente
Performance escrita	⚠ Custo médio (reorganiza)	⚠ Custo médio	✓ Menor (menos linhas)	✗ Lento (compressão)	✓✓ Muito rápido
Tamanho	Igual à tabela	Menor (chave + INCLUDE)	Menor (subset)	5–10× comprimido	Em memória (caro)
Disponível desde	Sempre	Sempre	SQL 2008	SQL 2012	SQL 2014
Recomendado para	Toda tabela com PK	Colunas de busca frequentes	Consultas frequentes por subset	Tabelas de DW (fact)	Cargas extremas (> 1M ops/s)
Cuidados	Chave clustered deve ser estreita	Não exagerar (degrada INSERT)	Planos ruins se estatísticas erradas	Não serve para OLTP	Persistência limitada

Regra geral: Toda tabela com PK deve ter `Clustered` na PK. Adicionar `Nonclustered` em colunas de busca frequentes. Considerar `Filtered` para subsets. `Columnstore` apenas em DW.

SINTAXE

Anatomia do CREATE INDEX

CREATE INDEX cria índices nonclustered. CREATE CLUSTERED INDEX cria o índice clustered. Cada parte da sintaxe tem impacto em performance.

COMPONENTES

- CREATE [UNIQUE] [CLUSTERED|NONCLUSTERED] INDEX — tipo
- nome — identificador (convenção: IX_Tabela_Colunas)
- ON tabela (colunas) — chave do índice
- INCLUDE (colunas) — colunas cobertas (não chave)
- WHERE condição — índice filtrado
- WITH (opções) — configurações

OPÇÕES IMPORTANTES

- ONLINE = ON — operação concorrente (Enterprise)
- FILLFACTOR = 80 — % de preenchimento (deixa espaço)
- PAD_INDEX = ON — aplica FILLFACTOR em níveis internos
- SORT_IN_TEMPDB = ON — sort em tempdb (acelera)
- DATA_COMPRESSION = PAGE — comprime (economiza I/O)
- MAXDOP = 4 — limita paralelismo
- DROP_EXISTING = ON — recria em vez de drop+create

BOAS PRÁTICAS

- Nome padronizado: IX_Tabela_Colunas
- Schema qualification: dbo.IX_...
- Esquema CREATE SCHEMA antes
- Considerar script de rollback

ANATOMIA_CREATE_INDEX.SQL — T-SQL · SQL Server

```
-- 1. Índice clustered (um por tabela)
CREATE CLUSTERED INDEX
 ON
 WITH
    ONLINE   ON
  FILLFACTOR     100
     DATA_COMPRESSION PAGE
 GO

-- 2. Índice nonclustered com INCLUDE (covering)
CREATE NONCLUSTERED INDEX
 ON
INCLUDE
 WITH
    ONLINE   ON
  FILLFACTOR     90
    PAD_INDEX  ON
 GO
-- Esta query usa covering index (sem Key Lookup):
-- SELECT Total, Status FROM dbo.Vendas
-- WHERE ClienteId = 42 AND DataVenda >= '2026-01-01'

-- 3. Índice filtrado (subset)
CREATE NONCLUSTERED INDEX
 ON
 WHERE          'ATIVO'     -- indexa apenas ativos
 GO
-- Útil quando 95% das queries filtram Status='ATIVO'

-- 4. Índice único
CREATE UNIQUE NONCLUSTERED INDEX
 ON
 WHERE      IS  NOT NULL
 GO

-- 5. Índice columnstore (DW)
CREATE CLUSTERED COLUMNSTORE INDEX
 ON
 GO

-- 6. Índice com computed column
CREATE NONCLUSTERED INDEX
 ON
INCLUDE
-- Onde ValorTotal = Quantidade * PrecoUnitario (persistido)
 GO

-- 7. Manutenção: verificar fragmentação
SELECT
          AS
                                  AS
                AS
 FROM
     DB_ID    NULL  NULL  NULL 'LIMITED'
 JOIN
     ON
     AND
 WHERE                                10
 ORDER BY                              DESC
 GO

-- 8. Rebuild (offline) ou Reorganize (online)
ALTER INDEX                          ON
    REBUILD WITH ONLINE   ON MAXDOP  4
-- OU:
ALTER INDEX                   ON          REORGANIZE
GO
```

STATS — A BASE DA OTIMIZAÇÃO

Estatísticas e Histograma

Estatísticas são histogramas de distribuição de valores em colunas. O otimizador as usa para estimar quantas linhas uma query vai retornar — base para escolha do plano. Estatística desatualizada = plano ruim.

TIPOS DE ESTATÍSTICA

- Auto-criadas — SQL Server cria automaticamente para colunas indexadas
- Manuais — `CREATE STATISTICS`
- Filter statistics — para índices filtrados
- Computed — em colunas calculadas

CONTEÚDO DE UMA ESTATÍSTICA

- Header — quando criada, linhas amostradas, densidade
- Density vector — seletividade por combinação de colunas
- Histograma — até 200 buckets (faixas de valores)

QUANDO ATUALIZAR

- Após grandes cargas (bulk insert, delete)
- Após mudanças de schema
- Periodicamente (job semanal)
- Quando plano subitamente ruim

COMANDOS

- `UPDATE STATISTICS tabela` — atualiza todas
- `UPDATE STATISTICS tabela WITH FULLSCAN` — amostra 100%
- `EXEC sp_updatestats` — todas as tabelas do DB
- `DBCC SHOW_STATISTICS` — ver conteúdo

AUTO-UPDATE

- Default ON (atualiza quando threshold atingido)
- Threshold: 20% de mudanças (tabelas grandes)
- SQL 2016+: Dynamic Statistics Auto-Update (mais rápido)
- Pode ser desativado (não recomendado)

ESTATISTICAS_HISTOGRAMA.SQL — T-SQL · SQL Server

```
-- 1. Ver estatísticas existentes
SELECT
            AS



        rowsAS
                    AS
                            AS
FROM
CROSSAPPLY
WHERE               OBJECT_ID  'dbo.Clientes'
GO

-- 2. Ver conteúdo completo (header, density, histograma)
DBCC                        'dbo.Clientes'     'PK_Clientes_ClienteId'
WITH
GO
-- Output:
-- range_hi_key | range_rows | eq_rows | distinct_range_rows | avg_range_rows
-- 1           | 0          | 1       | 0                   | 1
-- 1000        | 998        | 1       | 998                 | 1
-- 2000        | 999        | 1       | 999                 | 1

-- 3. Atualizar estatística
UPDASTEATISTICS
    WITHFULLSCAN    -- 100% das linhas (mais preciso)
GO

-- 4. Atualizar todas as estatísticas de uma tabela
UPDASTEATISTICS
GO

-- 5. Atualizar todas as estatísticas do database
EXEC
GO

-- 6. Criar estatística manual (coluna não-indexada)
CREATSTATISTICS
ON
WITHFULLSCAN
GO

-- 7. Forçar atualização com NORECOMPUTE
UPDASTEATISTICS
    WITHFULLSCANNORECOMPUTE
-- NORECOMPUTE: desabilita auto-update desta stat
-- (uso raro: quando estatística é crítica e se quer controle manual)
GO

-- 8. Ver estatísticas desatualizadas
SELECT
   OBJECT_NAME              AS
            AS

                            AS
        rowsAS
FROM
CROSSAPPLY
WHERE                       0
   AND                          rows   0.20  -- > 20% mudou
ORDERBY                     DESC
GO
```

CONCEITO CRÍTICO

SARGability

Predicados Otimizáveis

SARGable (Search Argument-able) é um predicado que permite ao otimizador usar um índice. Predicados não-SARGable forçam `Scan` (lê tudo) em vez de `Seek` (lê só o necessário).

PREDICADOS SARGABLE (✓)

- `WHERE coluna = valor`
- `WHERE coluna > valor`
- `WHERE coluna BETWEEN x AND y`
- `WHERE coluna IN (lista)`
- `WHERE coluna LIKE 'prefixo%'`
- `WHERE coluna IS NULL`

PREDICADOS NÃO-SARGABLE (✗)

- `WHERE UPPER(coluna) = 'X'` — função mata índice
- `WHERE YEAR(coluna_data) = 2026` — função mata índice
- `WHERE coluna LIKE '%substring%'` — curinga no início
- `WHERE coluna + 1 = 10` — aritmética
- `WHERE ISNULL(coluna, 0) = 0` — função
- `WHERE coluna = @var + 1` — expressão
- `WHERE CAST(coluna AS ...) = ...` — conversão

SOLUÇÕES

- **Function-based index**: índice em coluna calculada
- **Computed column persisted**: `NomeUpper AS UPPER(Nome) PERSISTED`
- **Range explícito**: `WHERE data >= '2026-01-01' AND data < '2027-01-01'`
- **Collation**: insensitive dispensa `UPPER()`
- **Avoid LIKE com `%prefixo`**

NÃO-SARGABLE	**Scan**	10.000 linhas = 10.000 leituras
SARGABLE	**Seek**	10.000 linhas com índice = ~5 leituras
DIFERENÇA	**100×–1000×**	mais rápido

SARGABLE_PREDICADOS.SQL — T-SQL · SQL Server

```
-- ❌ NÃO-SARGable: função na coluna mata índice
SELECT  FROM
WHERE UPPER  'JOÃO'
-- Plano: Clustered Index Scan (lê TODA a tabela)
-- Tempo: 1000ms para 100k linhas

-- ✅ SARGable: usar collation insensitive (default CI_AI)
SELECT  FROM
WHERE  'JOÃO'
-- Plano: Index Seek (lê só o necessário)
-- Tempo: 2ms
GO

-- ❌ NÃO-SARGable: função YEAR() na data
SELECT  FROM
WHERE YEAR  2026
-- Plano: Scan

-- ✅ SARGable: range explícito
SELECT  FROM
WHERE  '2026-01-01'
  AND  '2027-01-01'
-- Plano: Seek em índice de data
GO

-- ❌ NÃO-SARGable: LIKE com curinga no início
SELECT  FROM
WHERE  LIKE '%Santos%'
-- Plano: Scan

-- ✅ SARGable: LIKE com prefixo
SELECT  FROM
WHERE  LIKE 'Santos%'
-- Plano: Seek (usa índice em Nome)
GO

-- ❌ NÃO-SARGable: aritmética na coluna
SELECT  FROM
WHERE  1.1  100

-- ✅ SARGable: aritmética no literal
SELECT  FROM
WHERE  100  1.1
GO

-- Solução avançada: computed column persistida + índice
ALTER TABLE
ADD  AS UPPER  PERSISTED
GO

CREATE INDEX  ON
GO

-- Agora esta query usa índice:
SELECT  FROM
WHERE  'JOÃO'
-- Plano: Index Seek em IX_Clientes_NomeUpper
GO

-- ❌ NÃO-SARGable: ISNULL() na coluna
SELECT  FROM
WHERE ISNULL  'INATIVO'  'ATIVO'

-- ✅ SARGable: OR com IS NULL
SELECT  FROM
WHERE  'ATIVO' OR  IS NULL
GO
```

CONTROLE MANUAL

Hints — Quando Usar com Cautela

Hints são diretivas que forçam o otimizador a agir de uma forma específica — ignorando sua lógica de custo. Úteis em casos extremos, mas devem ser último recurso: o otimizador geralmente acerta.

TIPOS DE HINTS

- Query hints — `OPTION (...)` no final da query
- Table hints — `WITH (...)` após nome da tabela
- Join hints — `INNER LOOP JOIN`, `INNER HASH JOIN`
- Plan guides — aplicam hints a queries sem mudar código

QUERY HINTS COMUNS

- `OPTION (RECOMPILE)` — recompila a cada execução
- `OPTION (OPTIMIZE FOR UNKNOWN)` — usa estatística média
- `OPTION (OPTIMIZE FOR @var = valor)` — força valor específico
- `OPTION (MAXDOP n)` — limita paralelismo
- `OPTION (FAST n)` — retorna primeiras n linhas rápido
- `OPTION (HASH GROUP)` — força agregação via hash
- `OPTION (LOOP JOIN)` — força nested loops

TABLE HINTS

- `WITH (INDEX = IX_nome)` — força índice
- `WITH (NOLOCK)` — READ UNCOMMITTED (cuidado!)
- `WITH (READPAST)` — pula linhas bloqueadas
- `WITH (UPDLOCK, HOLDLOCK)` — lock de update mantido
- `WITH (ROWLOCK)` — força lock de linha

⚠ QUANDO USAR

- Parameter sniffing crônico (visto Aula 02)
- Otimizador escolhe plano ruim consistentemente
- Query específica não-usual
- Emergência de produção (parar o sangramento)

⚠ QUANDO NÃO USAR

- Sem ter tentado refatorar primeiro
- Sem estatísticas atualizadas
- Em queries comuns (o otimizador aprende)
- Como substituto de índice ausente

HINTS_QUANDO_USAR.SQL — T-SQL · SQL Server

```sql
-- 1. OPTION (RECOMPILE) — para parameter sniffing
CREATE OR ALTER PROCEDURE
    @ClienteId INT
AS
BEGIN
    SELECT * FROM
    WHERE @ClienteId
    OPTION (RECOMPILE)      -- recompila a cada execução
END
GO

-- 2. OPTION (OPTIMIZE FOR UNKNOWN) — usa estatística média
SELECT * FROM
WHERE @ClienteId
OPTION (OPTIMIZE FOR UNKNOWN)
GO

-- 3. OPTION (OPTIMIZE FOR valor) — força valor específico
SELECT * FROM
WHERE @ClienteId
OPTION (OPTIMIZE FOR (@ClienteId = 1))      -- otimiza para ID=1
GO

-- 4. OPTION (MAXDOP 1) — força single-thread
SELECT COUNT(*) FROM
OPTION (MAXDOP 1)
GO

-- 5. FORÇAR índice específico (table hint)
SELECT
FROM WITH (INDEX(
INNER JOIN ON
WHERE 'São Paulo'
GO

-- 6. WITH (NOLOCK) — permite dirty reads (CUIDADO!)
SELECT COUNT(*) FROM WITH (NOLOCK)
-- ⚠ Pode ler dados não committed (rollback posterior)
-- Útil apenas para relatórios aproximados em produção
GO

-- 7. WITH (UPDLOCK, HOLDLOCK) — para read-then-update seguro
BEGIN TRAN
SELECT * FROM
WHERE = 1
WITH (UPDLOCK, HOLDLOCK)    -- lock de update até fim da transação

-- ... outras operações ...

UPDATE SET = 100 WHERE = 1
COMMIT
GO

-- 8. Plan guides — aplicam hints sem mudar código (avançado)
EXEC
    @name = N'Guide_VendasPorCliente',
    @stmt = N'SELECT * FROM dbo.Vendas WHERE ClienteId = @Id',
    @type = N'SQL',
    @module_or_batch = NULL,
    @params = N'@Id INT',
    @hints = N'OPTION (RECOMPILE)';
-- Aplica o hint mesmo em código da aplicação que não podemos mudar
GO
```

CASOS REAIS

3 Casos Práticos de Otimização

Três cenários reais onde a aplicação das técnicas vistas transforma transforma queries lentas em rápidas.

CASOS

1. **Query com Key Lookup** — adicionar INCLUDE
2. **Estatística desatualizada** — plano muda completamente
3. **Predicado não-SARGable** — refatorar WHERE

RESULTADOS ESPERADOS

CASO 1	CASO 2	CASO 3
50×	**100×**	**200×**
mais rápido (elimina Key Lookup)	mais rápido (estatística correta)	mais rápido (Seek em vez de Scan)

CASOS_PRATICOS_OTIMIZACAO.SQL — T-SQL · SQL Server

```sql
-- ============ CASO 1: KEY LOOKUP ============
-- ❌ ANTES: Key Lookup degrada performance
SELECT
FROM
WHERE       'São Paulo'
-- Plano: Index Seek em IX_Clientes_Cidade
--        + Key Lookup para Buscar Nome, Email
-- Custo: 2 operadores, 5000 logical reads para 1000 linhas

-- ✅ DEPOIS: índice covering com INCLUDE
DROP INDEX          ON
CREATE NONCLUSTERED INDEX
ON
INCLUDE
-- Plano: Index Seek único (sem Key Lookup)
-- Custo: 1 operador, 5 logical reads
-- Ganho: ~1000x menos I/O
GO

-- ============ CASO 2: ESTATÍSTICA DESATUALIZADA ============
-- ❌ ANTES: estatística velha gera plano ruim
-- 1 milhão de vendas em 30 dias, mas estatística
-- mostra '1000 vendas em 30 dias' (desatualizada)
SELECT          COUNT  AS
FROM
INNER JOIN          ON
WHERE           '2026-09-01'
GROUP BY
-- Plano: Nested Loops (otimizador achou que eram poucas linhas)
-- Tempo: 45000ms (45 segundos)

-- ✅ DEPOIS: atualizar estatística
UPDATE STATISTICS
  WITH FULLSCAN
-- Mesma query, novo plano:
-- Plano: Hash Match Join (otimizador viu 1M de linhas)
-- Tempo: 850ms (53x mais rápido!)
GO

-- ============ CASO 3: PREDICADO NÃO-SARGABLE ============
-- ❌ ANTES: função na coluna mata índice
SELECT
FROM
WHERE YEAR          2026
  AND MONTH         9
-- Plano: Clustered Index Scan (lê TODAS as vendas)
-- Tempo: 12000ms (12 segundos) para 10M de linhas

-- ✅ DEPOIS: range explícito SARGable
SELECT
FROM
WHERE         '2026-09-01'
  AND         '2026-10-01'
-- Plano: Index Seek em IX_Vendas_DataVenda
-- Tempo: 60ms (200x mais rápido!)
GO

-- ============ BÔNUS: DELETE EM LOTE SEM LOCKS LONGOS ============
-- ❌ ANTES: DELETE único de milhões de linhas
DELETE FROM
WHERE         '2025-01-01'
-- Problema: lock exclusivo por horas, log transacional gigante

-- ✅ DEPOIS: DELETE em lotes (batch)
WHILE 1  1
BEGIN
    DELETE TOP 5000 FROM
     WHERE      '2025-01-01'
    IF @@ROWCOUNT    0 BREAK
    WAITFOR DELAY '00:00:00.1'      -- pausa para reduzir pressão
END
-- Vantagem: locks curtos, log pequeno, retomável
GO
```

BLOCO 7.2 / 2

7.2

~ 50 MIN · 9 TÓPICOS

CHAPTER

Ferramentas de Monitoramento e Análise de Desempenho no SQL Server

O arsenal de diagnóstico — DMVs, Query Store, Extended Events, Wait Stats.

01 → DMVs essenciais

02 → Statistics IO/TIME

03 → Query Store

04 → Extended Events

05 → Wait statistics

06 → Activity Monitor

07 → Performance Monitor

08 → Profiler vs XEvents

09 → Checklist de diagnóstico

DMVS — DYNAMIC MANAGEMENT VIEWS

DMVs Essenciais para Diagnóstico

DMVs são visões de sistema que expõem estado interno do SQL Server. Em Server. Em tempo real, mostram queries em execução, locks, waits, planos, planos, estatísticas. São a ferramenta nº1 de diagnóstico.

CATEGORIAS DE DMVS

- ›sys.dm_exec_* — execução (queries, requests, connections)
- ›sys.dm_os_* — sistema operacional (waits, memory, schedulers)
- ›sys.dm_tran_* — transações (locks, enqueues)
- ›sys.dm_db_* — database (índices, estatísticas, filestream)
- ›sys.dm_io_* — I/O (virtual file stats, buffer pool)

TOP DMVS POR USO

- ›dm_exec_requests
- ›dm_exec_query_stats
- ›dm_exec_sql_text
- ›dm_exec_query_plan
- ›dm_os_wait_stats
- ›dm_tran_locks
- ›dm_os_waiting_tasks
- ›dm_db_index_usage_stats
- ›dm_db_index_physical_stats

PERMISSÕES

VIEW SERVER STATE
maioria das DMVs

VIEW DATABASE STATE
DMVs de database

⚠ **Atenção:** Resetam no restart do serviço. Para histórico persistente, usar persistente, usar Query Store.

✦ T-SQL · 5 QUERIES REPRODUZÍVEIS

SSMS ·

```
-- 1. TOP 20 queries mais caras por CPU
SELECT TOP 20
  SUBSTRING
      CASE
        WHEN 1 DATALENGTH
        ELSE
      END                    2 1 AS
                  AS                AS
                1000                  AS
                1000                  AS
                AS
FROM sys.dm_exec_query_stats
CROSS APPLY sys.dm_exec_sql_text
WHERE      DB_ID
ORDER BY          DESC
GO

-- 2. Requests em execução AGORA
SELECT


           1000 AS
           1000 AS
                1000 AS
        AS
  DB_NAME       AS


FROM sys.dm_exec_requests
INNER JOIN sys.dm_exec_sessions
CROSS APPLY sys.dm_exec_sql_text
WHERE          @@SPID
  AND                1
ORDER BY             DESC
GO

-- 3. Quem está bloqueando quem?
SELECT
          AS
               AS
         1000 AS
       AS
FROM sys.dm_exec_requests
OUTER APPLY sys.dm_exec_sql_text
WHERE                          0
ORDER BY       DESC
GO

-- 4. Índices não utilizados (candidatos a drop)
SELECT
 OBJECT_NAME      AS
     AS
       AS

FROM sys.indexes
LEFT JOIN sys.dm_db_index_usage_stats
   ON
   AND
   AND          DB_ID
WHERE                0   0
  AND
  OBJECT_NAME      NOT LIKE 'sys%'
  AND COALESCE                    0 0
ORDER BY       DESC -- updates sem leituras = custo inútil
GO

-- 5. Índices faltantes sugeridos pelo otimizador
SELECT TOP 10
   ROUND                0 AS
                  AS
              AS
              AS
              AS
 OBJECT_NAME      AS
FROM sys.dm_db_missing_index_details
INNER JOIN sys.dm_db_missing_index_groups
   ON
INNER JOIN sys.dm_db_missing_index_group_stats
   ON
ORDER BY       DESC
GO
```

✦ MÉTRICAS DETALHADAS

SET STATISTICS IO e TIME

Quando você executa uma query com `SET STATISTICS IO ON` e `TIME ON`, o SQL SQL Server imprime no output métricas detalhadas de I/O e tempo. Essencial para para comparar performance antes/depois de otimizações.

O QUE CADA MÉTRICA SIGNIFICA

- logical readspáginas lidas do buffer pool (memória). Métrica mais importante. importante.
- physical readspáginas lidas do disco (na hora)
- read-ahead readspáginas pré-carregadas (prefetch)
- lob logical readsleituras de LOB (text, image, varchar(max))
- scan countnúmero de scans/seeks
- CPU timemilissegundos de processamento
- elapsed timetempo total (incluindo waits)

COMO INTERPRETAR

- ›logical reads alto → índice ausente ou ineficiente
- ›physical reads alto → buffer pool frio (dados não em RAM)
- ›read-ahead alto → otimização de prefetch funcionando
- ›scan count > 0 → múltiplas buscas (pode ser paralelismo)
- ›CPU time > elapsed → paralelismo (múltiplas CPUs)
- ›elapsed >> CPU → muitos waits (locks, I/O)

✓ PADRÃO PROFISSIONAL

- Medir ANTES e DEPOIS de cada otimização
- Comparar logical reads (métrica nº1)
- Documentar ganho (ex: 5000 → 50 reads = 100x)

✦ T-SQL · ANTES VS DEPOIS

GAIN: 287X READS · 83X CPU

```
-- 1. Habilitar métricas
SET           ON
SET             ON
GO

-- 2. Query para medir
SELECT      COUNT  AS          SUM      AS
FROM
INNERJOIN
   ON
WHERE        'ATIVO'
  AND          '2026-01-01'
GROUPBY
HAVINGSUM         1000
ORDERBY     DESC
GO

-- Output típico (ANTES da otimização):
--
-- Table 'Vendas'. Scan count 5, logical reads 15420,
--   physical reads 8, read-ahead reads 15200,
--   lob logical reads 0, lob physical reads 0,
-- lob read-ahead reads 0.
--
-- Table 'Clientes'. Scan count 1, logical reads 89,
--   physical reads 0, read-ahead reads 85,
--   lob logical reads 0, lob physical reads 0,
-- lob read-ahead reads 0.
--
-- SQL Server Execution Times:
--   CPU time = 1250 ms,  elapsed time = 1320 ms.

-- ============ APÓS OTIMIZAÇÃO (criar índices) ============
CREATE          INDEX
ON
INCLUDE
GO

-- Mesma query:
-- Table 'Vendas'. Scan count 1, logical reads 42,
--   physical reads 0, read-ahead reads 38.
-- Table 'Clientes'. Scan count 1, logical reads 12,
--   physical reads 0, read-ahead reads 10.
--
-- SQL Server Execution Times:
--   CPU time = 15 ms,  elapsed time = 18 ms.

-- Comparativo:
--   logical reads: 15509 → 54 (287x menos!)
--   CPU time:      1250ms → 15ms (83x mais rápido)
--   elapsed time:  1320ms → 18ms (73x mais rápido)

-- 3. Desabilitar métricas
SET           OFF
SET             OFF
GO

-- 4. Alternativa: medir em variáveis
DECLARE@Start      SYSDATETIME

SELECT      COUNT  AS          SUM      AS
FROM
INNERJOIN          ON
WHERE        'ATIVOAND          '2026-01-01'
GROUPBY
HAVINGSUM         1000

PRINT'Duração: '
   CASTDATEDIFF         @StartSYSDATETIME AS
GO
```

QUERY STORE — RECURSO CRÍTICO

Query Store — O Black Box

Adicionado no SQL Server 2016, o Query Store é o recurso mais importante para para diagnóstico de performance. Captura e persiste histórico de queries, planos, waits e planos, waits e runtime stats — sobrevive a restarts. Substitui ferramentas legadas. legadas.

O QUE CAPTURA

- ›Todas as queries executadas
- ›Planos de execução (múltiplas versões)
- ›Runtime statistics (CPU, reads, duration)
- ›Wait statistics por query
- ›Histórico temporal (quando cada plano foi usado)

VANTAGENS

- ›Persistente (sobrevive a restarts)
- ›Automático (sem config adicional)
- ›Baixo overhead (1-3% em cargas típicas)
- ›Permite forçar planos bons
- ›Diagnóstico de regressões

CONFIGURAÇÃO

`ALTER DATABASE SET QUERY_STORE = ON` Default SQL 2022+ · Azure SQL

Histórico 30 dias Tamanho 100MB → 1GB+)

◆ CASOS DE USO & RECURSOS

- "A query que era rápida ficou lenta" — comparar planos
- Forçar plano bom após estatística desatualizada
- Identificar top queries consumidoras
- Análise de regressão após deploy
- Query Store Hints (SQL 2022+) · Automatic Tuning (Azure SQL) · relatórios SSMS

✦ T-SQL · 5 SNIPPETS QUERY STORE

PERSISTENTE · SOBREVIVE A

```
-- 1. Habilitar Query Store (se ainda não estiver)
ALTER DATABASE
SET ON
                                    30
                  900
            1000
                  60

GO

-- 2. Verificar status
SELECT



FROM sys.database_query_store_options
GO

-- 3. TOP 10 queries por CPU média
SELECT TOP 10

                1000 AS
                1000 AS
                     AS
                  AS
FROM sys.query_store_query
INNER JOIN sys.query_store_query_text
   ON
INNER JOIN sys.query_store_plan
   ON
INNER JOIN sys.query_store_runtime_stats
   ON
ORDER BY          DESC
GO

-- 4. Detectar regressão: query que ficou lenta
SELECT

           AS
           AS
               1000 AS
               1000 AS
                              1000
     AS
FROM sys.query_store_query
INNER JOIN sys.query_store_query_text
   ON
INNER JOIN sys.query_store_plan
   ON
INNER JOIN sys.query_store_runtime_stats
   ON
INNER JOIN sys.query_store_plan
   ON
   AND
INNER JOIN sys.query_store_runtime_stats
   ON
WHERE                                5 -- 5x pior
ORDER BY       DESC
GO

-- 5. Forçar plano bom (reverte regressão instantaneamente)
EXEC sp_query_store_force_plan
  @query_id 42
  @plan_id 87
GO
-- Remove a força:
-- EXEC sp_query_store_unforce_plan @query_id = 42, @plan_id = 87;
```

✦ CAPTURA DE EVENTOS

Extended Events e Wait Statistics

Extended Events (XEvents) é a ferramenta moderna de captura de eventos — substitui SQL substitui SQL Profiler (deprecated). Captura deadlocks, blocking, long queries, waits. Wait waits. Wait Statistics mostra onde o SQL Server está gastando tempo esperando. esperando.

EXTENDED EVENTS

- ›Substituto do Profiler (legado, deprecated)
- ›Muito mais leve (1% do overhead do Profiler)
- ›Persistente em arquivo .xel ou ring buffer
- ›Captura eventos específicos (deadlock, block, long query)
- ›Padrão de produção para auditoria

EVENTOS ÚTEIS

- ›xml_deadlock_report — deadlocks
- ›blocked_process_report — blocking
- ›sql_statement_completed — queries (com filtro)
- ›long_query · module_end · error_reported

WAIT STATISTICS · TOP WAIT TYPES

LCK_M_* esperando locks	PAGEIOLATCH_* esperando I/O
ASYNC_NETWORK_IO cliente lento	SOS_SCHEDULER_YIELD CPU pressure
WRITELOG esperando log	CXPACKET paralelismo

✦ PADRÃO DE DIAGNÓSTICO

1 Capturar waits → 2 Top wait type → 3 Investigar causa → 4 Aplicar correção

✦ T-SQL · 5 SNIPPETS XEVENTS + WAITS .XEL ·

```
-- 1. Criar sessão XEvent para capturar deadlocks
CREATE EVENT SESSION
ON SERVER
ADD EVENT sqlserver.xml_deadlock_report
  ACTION
      sqlserver.client_app_name
      sqlserver.client_hostname
      sqlserver.sql_text
      sqlserver.username

ADD TARGET package0.event_file
   SET       N'C:\Temp\deadlocks.xel'
                50  5  MB

WITH
          4096
             ON -- inicia com o servidor

ALTER EVENT SESSION        ON SERVER STATE
GO

-- 2 Capturar queries longas (>1s)
CREATE EVENT SESSION
ON SERVER
ADD EVENT sqlserver.sql_statement_completed
  ACTION
      sqlserver.client_app_name
      sqlserver.client_hostname
      sqlserver.sql_text
      sqlserver.username
      sqlserver.database_name
   WHERE      1000000 -- microssegundos (1s)
ADD TARGET package0.event_file
   SET       N'C:\Temp\queries_longas.xel'
               100 10

WITH          ON
ALTER EVENT SESSION        ON SERVER STATE
GO

-- 3. TOP 20 wait types acumulados
SELECT TOP 20
                AS
           1000.0 AS
          NULLIF                0 1000.0
     AS
           1000.0 AS
                      1000.0
   AS
  CAST      100.0
     SUM       OVER AS      52
    AS
FROM sys.dm_os_wait_stats
WHERE     NOT IN
    -- waits irrelevantes (system)
  'SLEEP_TASK''BROKER_TASK_STOP''BROKER_TO_FLUSH'
    'CHECKPOINT_QUEUE''DISPATCHER_QUEUE
    'FT_IFTS_SCHEDULER_IDLE_WAIT''HADR_WORK_QUEUE'
    'XE_DISPATCHER_WAIT''XE_TIMER_EVENT'
    'CLR_AUTO_EVENT''PAGE_HADR_WAIT''FILESTREAM_IOMGR_IOCOMPLETION'
    'DISPATCHER_QUEUE_SEMAPHORE''SERVER_IDLE_CHECK''TRANSMITTER'
    'REQUEST_FOR_DEADLOCK_SEARCH''SERVER_DIAGNOSTICS_SLEEP'
    'LAZYWRITER_SLEEP''SLEEP_BPOOL_FLUSH''PAGE_POLL'
  'WAITFOR''BROKER_RECEIVE_WAITFOR'
    0
  AND
ORDER BY      DESC
GO

-- 4. Ler eventos capturados (deadlocks)
SELECT
  DATEADD DATEDIFF GETUTCDATE GETDATE
              (event/@timestamp) 'dd/mm/yyyy hh:mi:ss'
     AS
       '(event/data[@name="victim_process_id"]/value)[1]'
     AS
   CAST     '(event/data[@name="deadlock_graph"]/value/deadlock)[1]'
     AS   AS
FROM sys.fn_xe_file_target_read_file
          'C:\Temp\deadlocks*.xel' NULL NULL
CROSS APPLY      event AS
GO

-- 5. Resetar wait stats (após baseline)
DBCC SQLPERF('sys.dm_os_wait_stats')
GO
```

GUI · MONITORAMENTO

Ferramentas Visuais de Monitoramento

Quatro ferramentas complementares — cada uma com papel específico.

Activity Monitor · Performance Monitor · SQL Profiler · Extended Events

Característica	Activity MonitorGUI no SSMS	Performance MonitorWindows OS tool	SQL ProfilerGUI legada (deprecated)	Extended EventsModerno (T-SQL + GUI)
Tipo	GUI no SSMS	Windows OS tool	GUI legada (deprecated)	Moderno (T-SQL + GUI)
Foco	Estado instantâneo	Contadores históricos	Captura de eventos	Captura moderna
Persistência	Em memória (não persiste)	Log file .blg	Trace file .trc	Event file .xel
Performance	✓ Leve	✓ Muito leve	✗ Alto overhead	✓✓ Muito leve
Granularidade	Por sessão/request	Por contador	Por evento	Por evento
Histórico	Apenas tempo real	✓ Configurável	✓ Trace file	✓ Event file
Alertas	✗ Não	✓ Sim (email, script)	⚠ Limitado	✓ Via SQL Agent
Disponível desde	SQL 2005	Sempre (Windows)	SQL 7.0 (deprecated)	SQL 2008
Recomendado	✓ Quick check	✓ Coleta sistemática	✗ Não (deprecated)	✓✓ Produção
Quando usar	"O que está acontecendo agora?"	Coleta sistemática 24/7	Legado (migrar!)	Captura de eventos
Atalho SSMS	Ctrl+ Alt+ A	perfmon.msc	Tools > SQL Profiler	Management > Extended Events
Caso típico	Quick diagnose em incidente	Dashboard 24/7	Sistemas antigos	Deadlocks, long queries, blocking

◆ Regra geral: Activity Monitor para "o que está acontecendo agora?", Performance Monitor para coleta sistemática, Extended Events para captura específica. NUNCA usar SQL Profiler em produção.

✓ DIAGNÓSTICO PROFISSIONAL

Checklist Profissional de Diagnóstico de Query Lenta

Seis passos para diagnosticar qualquer query lenta em produção.

RESOLVIDOS NOS PASSOS
1-3
80% DOS CASOS

01 Medir com STATISTICS IO/TIME

Sempre comece medindo: `SET STATISTICS IO ON; SET STATISTICS TIME ON`. Compare logical reads e CPU time antes/depois. Não otimize sem medir.

TOOL `SET STATISTICS IO, TIME ON`

02 Ler Plano de Execução

`Ctrl+M` no SSMS para plano real. Procure o operador mais mais grosso (maior custo). Table Scan em tabela grande? grande? Falta índice. Key Lookup? Adicionar INCLUDE. INCLUDE.

TOOL `SET STATISTICS XML ON` · SSMS Ctrl+M

03 Verificar Estatísticas

Estatística desatualizada gera plano ruim. Compare EstimateRows vs ActualRows no plano. Discrepância grande? grande? `UPDATE STATISTICS WITH FULLSCAN`.

TOOL `DBCC SHOW_STATISTICS` · `sys.dm_db_stats_prope`

04 Analisar SARGability

Função na coluna do WHERE mata índice. `YEAR(data)= 2026`? Refatorar para range. `UPPER(Nome)='X'`? Usar collation insensitive. Ganho de de 100x+.

TOOL Revisão manual do WHERE

05 Identificar Wait Type

Query lenta mas sem CPU alta? Provavelmente está esperando (wait). Capturar wait type via `sys.dm_os_waiting_tasks` ou XEvents. LCK_M_* = locks, PAGEIOLATCH = I/O.

TOOL `sys.dm_os_wait_stats` · `sys.dm_os_waiting_`

06 Confirmar com Query Store

Query que era rápida e ficou lenta? Query Store mostra mostra histórico de planos. Identifique regressão e force force plano bom via `sp_query_store_force_plan`.

TOOL `sys.query_store_*` views

✦ Regra de ouro: nunca otimize sem medir. 80% dos problemas são resolvidos nos passos 1-3.

6 passos · qualquer query lenta

✦ SÍNTESE DA AULA 07

O que vimos hoje

7.1 PLANOS DE EXECUÇÃO & ÍNDICES

1. 1. Planos de Execução — estimado vs real, operadores (Scan, Seek, Join, Sort)
2. 2. Tipos de Índice — clustered, nonclustered, INCLUDE, columnstore
3. 3. Estatísticas — histograma, EstimateRows vs ActualRows
4. 4. SARGability — funções na coluna matam índice

7.2 FERRAMENTAS DE MONITORAMENTO

1. 5. DMVs essenciais — sys.dm_exec_*, sys.dm_os_*, sys.dm_db_*
2. 6. SET STATISTICS IO/TIME — logical reads é métrica nº1
3. 7. Query Store — regressão, forçar plano, persistente
4. 8. Extended Events + Wait Stats + ferramentas visuais + checklist 6 passos

Próximos Passos

- ▶ Reproduzir todos os códigos no SSMS local com AdventureWorks2019 AdventureWorks2019
- ▣ Habilitar Query Store em TODAS as DBs de produção
- ✓ Aplicar checklist de 6 passos em uma query lenta real
- ✦ Configurar Extended Events para deadlocks
- ⌕ Revisar queries existentes para SARGability
- ▿ Estudar para certificação DP-300 (Azure Database Administrator)
- ▤ Ler: 'SQL Server 2019 Query Performance Tuning' (Grant Fritchey)

✦ DESAFIO FINAL

```
-- Desafio final:
-- 1. Identifique a query mais lenta em produção
--    (top 1 por avg_duration no Query Store)
-- 2. Aplique o checklist de 6 passos
-- 3. Documente: antes (logical_reads, cpu_ms)
-- 4. Otimize (índice, estatística, SARGable)
-- 5. Meça depois e calcule ganho
-- 6. Apresente em 5 min na próxima aula
```

PRÓXIMA AULA

→ Aula 08: High Availability · AlwaysOn · Backup/Restore · Disaster Recovery Recovery